

Table of Content

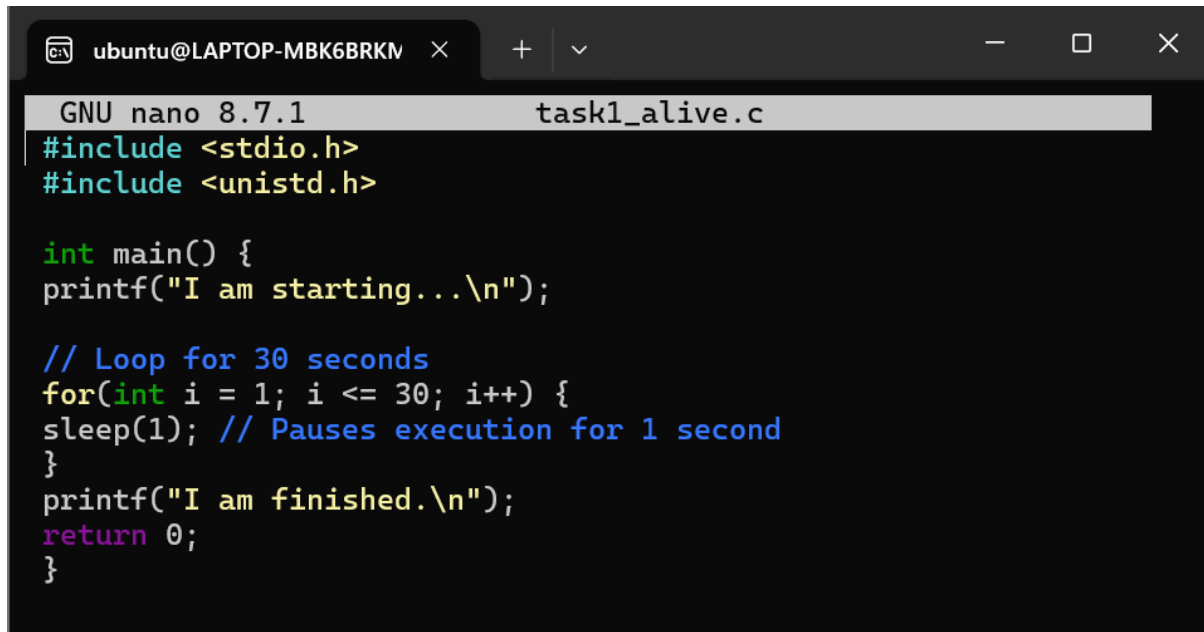
II. Task 1: The Long-Running Process	3
A. C Source Code	3
B. Terminal Execution and Verification	3
III. Task 2: Process Identity(PID and PPID)	4
A. C Source Code	4
B. Terminal Execution and Verification	5
IV. Task 3: Exit Codes and OS Feedback	5
A. C Source Code	6
B. Terminal Execution and Verification	6
V. Task 4: Standard I/O Streams	7
A. C Source Code	7
B. Terminal Execution	7
VI. Task 5: Conditional Execution and Termination	8
A. C Source Code	8
B. Terminal Execution and Verification	9

Table of Figure

Figure 1A: task1_alive.c.....	3
Figure 1B: Terminal 1: Run in Background.....	3
Figure 1B: Terminal 2: Monitor the process.....	4
Figure 2A: task2_idenitfy.c.....	4
Figure 2B: Terminal Commands.....	5
Figure 3A: task3_exit.c.....	6
Figure 3B: checking Exit Status.....	7
Figure 4A: Task4_input.c.....	7
Figure 4B: Running task4.....	7
Figure 5A: task5_control.c.....	8
Figure 5B: Testing Control Flow.....	9

II. Task 1: The Long-Running Process

A. C Source Code



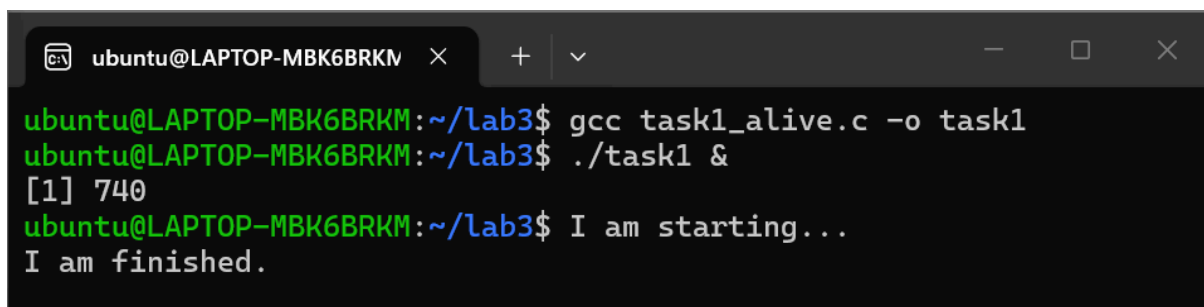
```
GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
printf("I am starting...\n");

// Loop for 30 seconds
for(int i = 1; i <= 30; i++) {
sleep(1); // Pauses execution for 1 second
}
printf("I am finished.\n");
return 0;
}
```

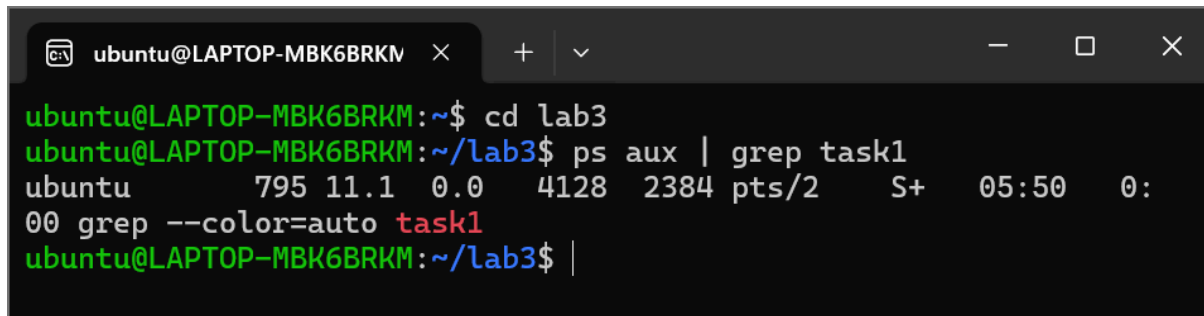
Figure 1A: task1_alive.c

B. Terminal Execution and Verification



```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ gcc task1_alive.c -o task1
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task1 &
[1] 740
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ I am starting...
I am finished.
```

Figure 1B: Terminal 1: Run in Background

A terminal window titled 'ubuntu@LAPTOP-MBK6BRKM' with standard window controls. The user has navigated to the 'lab3' directory and executed the command 'ps aux | grep task1'. The output shows a single process: '00 grep --color=auto task1' with PID 795, PPID 11.1, and other system details.

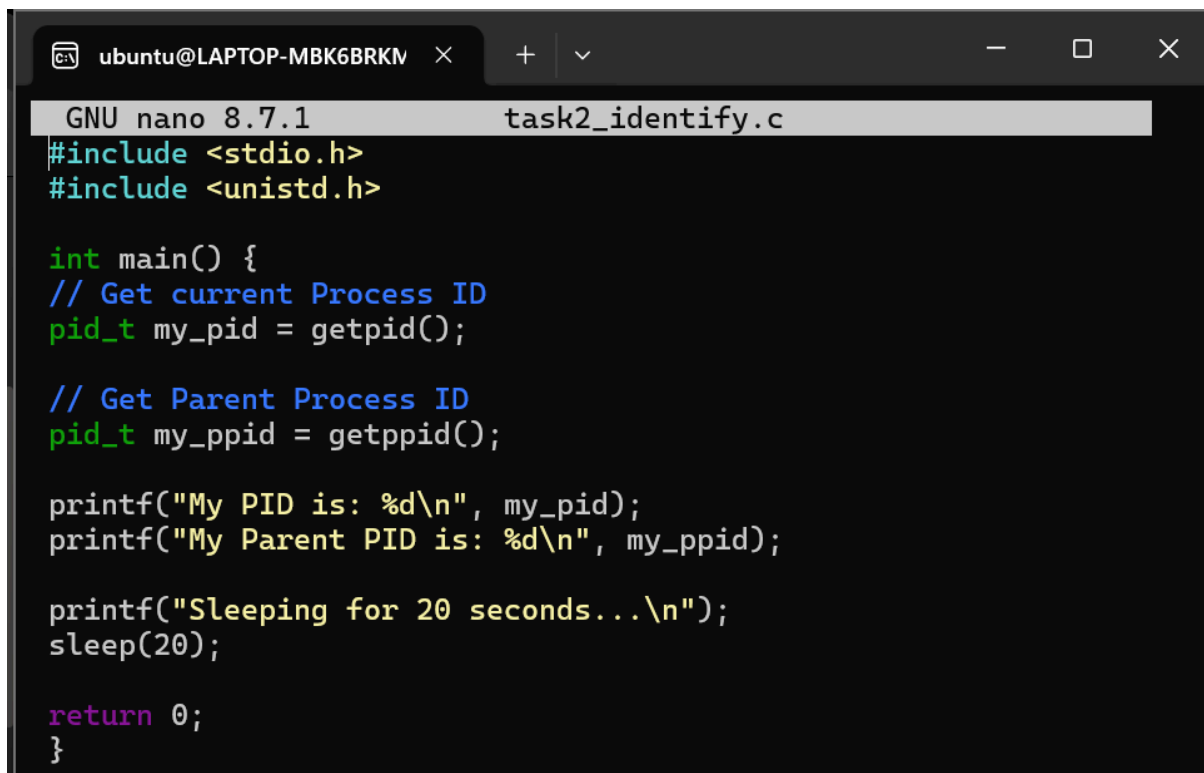
```
ubuntu@LAPTOP-MBK6BRKM:~$ cd lab3
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ps aux | grep task1
ubuntu      795 11.1  0.0   4128  2384 pts/2    S+   05:50   0:
00 grep --color=auto task1
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 1B: Terminal 2: Monitor the process

Summary: In Task 1, I used nano to write code, then used gcc to compile it into an executable, run the code, and check the result. Additionally used (ps aux | grep) in a separate terminal, which let me know which process was being used and its details.

III. Task 2: Process Identity(PID and PPID)

A. C Source Code

A screenshot of the GNU nano 8.7.1 text editor showing the source code for 'task2_identify.c'. The code includes headers for stdio and unistd, defines a main function that uses getpid() and getppid() to retrieve the current and parent process IDs, prints them out, sleeps for 20 seconds, and then returns 0.

```
GNU nano 8.7.1 task2_identify.c
#include <stdio.h>
#include <unistd.h>

int main() {
// Get current Process ID
pid_t my_pid = getpid();

// Get Parent Process ID
pid_t my_ppid = getppid();

printf("My PID is: %d\n", my_pid);
printf("My Parent PID is: %d\n", my_ppid);

printf("Sleeping for 20 seconds...\n");
sleep(20);

return 0;
}
```

Figure 2A: task2_identify.c

B. Terminal Execution and Verification

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ gcc task2_identify.c -o task2
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task2 &
[1] 902
My PID is: 902
My Parent PID is: 321
Sleeping for 20 seconds...
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task2 &
[1] 974
My PID is: 974
My Parent PID is: 321
Sleeping for 20 seconds...
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ps -p 974 -o pid,ppid,cmd
  PID   PPID  CMD
   974    321 ./task2
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 2B: Terminal Commands

Summary: In Task2, I mainly used the “./ task2 &” command in the terminal, which helped the code to run in the background and showed the unique ID (PID) provided to every process. Additionally, using “ps -p 974 -o pid, ppid, cmd ” helped to know about the PPID and CMD, which is the parent of the PID and the folder name.

IV. Task 3: Exit Codes and OS Feedback

A. C Source Code

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ nano task2_exit.c
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ cat task2_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success,
    negative for fail): ");
    scanf("%d", &num);
    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

Figure 3A: task3_exit.c

B. Terminal Execution and Verification

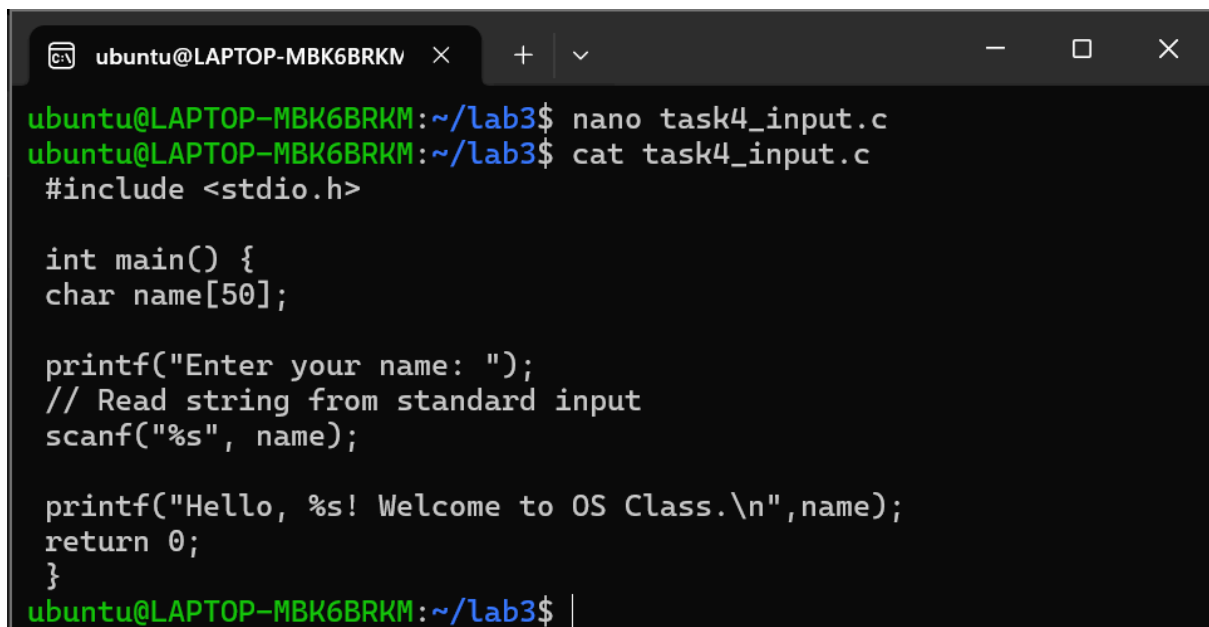
```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ nano task3_exit.c
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ gcc task3_exit.c -o task3
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ echo $?
0
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ echo $?
1
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 3B: checking Exit Status

Summary: Repetition of task 1 and task2, but there is a new process which is used in task 3, which is "echo \$?", which is a special variables which shows the exit status code which shows the "0" for success and "1" for failure

V. Task 4: Standard I/O Streams

A. C Source Code

A terminal window titled 'ubuntu@LAPTOP-MBK6BRKM' with standard window controls. The terminal shows the following commands and output:

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ nano task4_input.c
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ cat task4_input.c
#include <stdio.h>

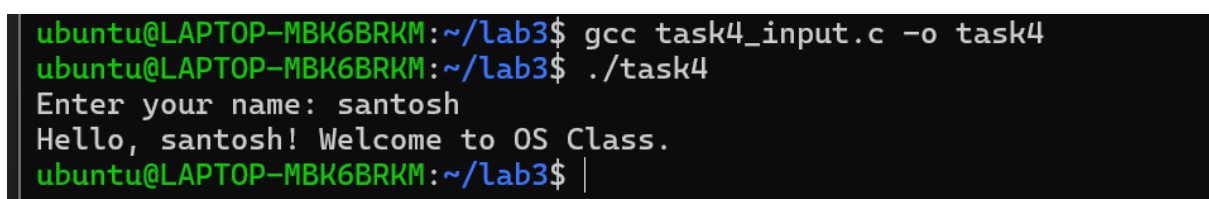
int main() {
char name[50];

printf("Enter your name: ");
// Read string from standard input
scanf("%s", name);

printf("Hello, %s! Welcome to OS Class.\n",name);
return 0;
}
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 4A: Task4_input.c

B. Terminal Execution

A terminal window titled 'ubuntu@LAPTOP-MBK6BRKM' with standard window controls. The terminal shows the following commands and output:

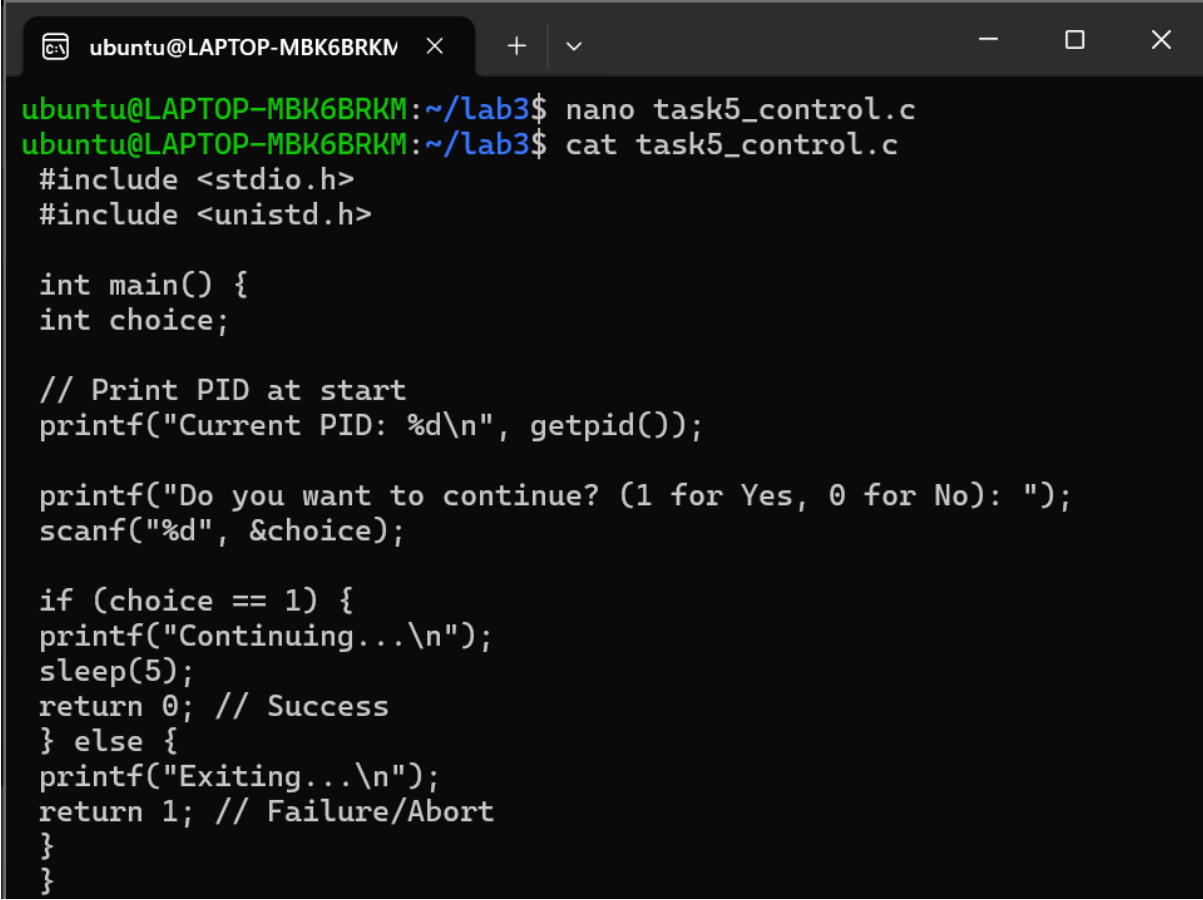
```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ gcc task4_input.c -o task4
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task4
Enter your name: santosh
Hello, santosh! Welcome to OS Class.
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 4B: Running task4

Summary: In task4, Used the code which can interact with the user and give output through the terminal. Using printf and scanf.

VI. Task 5: Conditional Execution and Termination

A. C Source Code

A terminal window with a dark background and light-colored text. The window title bar shows 'ubuntu@LAPTOP-MBK6BRKM' and standard window controls. The terminal content shows the user creating and displaying the source code for 'task5_control.c'. The code includes standard headers, a main function that prints the PID, asks for a choice to continue, and uses an if-else statement to either sleep for 5 seconds or exit with an error code.

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ nano task5_control.c
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ cat task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
int choice;

// Print PID at start
printf("Current PID: %d\n", getpid());

printf("Do you want to continue? (1 for Yes, 0 for No): ");
scanf("%d", &choice);

if (choice == 1) {
printf("Continuing...\n");
sleep(5);
return 0; // Success
} else {
printf("Exiting...\n");
return 1; // Failure/Abort
}
}
```

Figure 5A: task5_control.c

B. Terminal Execution and Verification

```
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ gcc task5_control.c -o task5
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task5
Current PID: 1794
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ echo $?
0
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ ./task5
Current PID: 1815
Do you want to continue? (1 for Yes, 0 for No): no
Exiting...
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ echo $?
1
ubuntu@LAPTOP-MBK6BRKM:~/lab3$ |
```

Figure 5B: Testing Control Flow